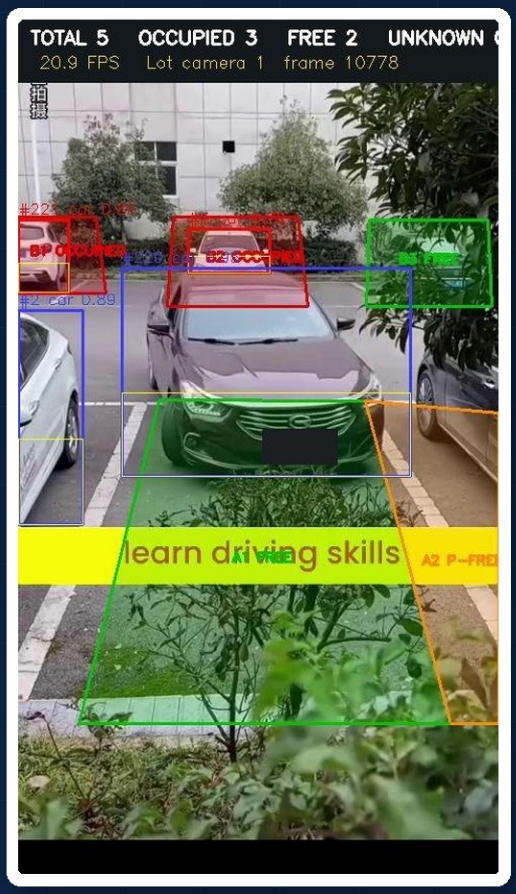
PERSONAL PROJECT · TECHNICAL REPORT

AI-Based Smart Parking Management System

Real-time vehicle detection, tracking and occupancy analytics for camera-monitored parking

BUILT WITH

Python 3.10 | FastAPI | YOLO11s
ByteTrack | PP-OCRv4 | SQLite
React 19 | TypeScript | Docker Compose (scaffold)



LIVE LOT CAMERA · ANNOTATED OUTPUT
Real street clip served over MJPEG. Registration plate covered; ANPR output disabled for this stream.

AUTHOR

Saurabh V Patil

Personal Project · October 2026

github.com/saurabh876724/Smart-parking

SECTION 01 · OVERVIEW

1 Project Overview

The AI-Based Smart Parking Management System is a real-time computer-vision platform that turns ordinary fixed cameras into parking sensors. **One camera per zone is enough:** the system detects and tracks vehicles, computes per-bay occupancy, recognises plates at the gate, records parking sessions, and exposes everything through a REST/WebSocket API and a live React dashboard.

31 + 2	646	6	9
REST routes + WebSocket routes under /api/v1	backend tests passed (recorded run, 2026-10-14)	architecture layers, import-isolated by guard tests	database tables, managed by Alembic migrations

What it does

Live occupancy

A YOLO11s detector and ByteTrack tracker feed a per-slot state machine, so the dashboard always shows which bays are free, occupied or unknown.

Gate sessions

A second camera detects line crossings, reads the plate with an ANPR chain (PP-OCRv4 OCR + validation + voting) and binds it to a parking session with duration and status.

Audited API

31 REST routes and 2 WebSocket routes under /api/v1, role-based access, plate masking and an audit log of every action.

Live dashboard

SVG slot map, session table and annotated MJPEG camera stream, all updating without a page refresh.

Problem it solves

- Manual counting and paper logs are slow, error-prone and give no live occupancy picture.
- Sensor-per-bay systems (magnetic, ultrasonic) are expensive to install and maintain across a whole lot.
- Small and mid-size lots get no session history, no entry/exit automation and no usage audit without dedicated staff.

Why it is useful

- One commodity camera replaces per-bay hardware; new zones are added with configuration, not construction.
- Operators see occupancy, history and live video in one place instead of walking the lot.
- Storage (SQLite/PostgreSQL by URL), camera sources (file/webcam/RTSP) and roles are config-driven, so the core pipeline is unchanged between deployments.

Main objectives

- Live per-bay occupancy from a standard CCTV feed, using calibrated slot polygons per camera.
- Stable vehicle identity across frames, so occupancy is reasoned about per track rather than per detection.
- Gate entry/exit detection with plate capture, producing parking sessions that can be searched and reviewed.
- A secured, role-based API and a live dashboard that updates over WebSockets instead of polling.
- A modular, testable codebase whose claims are verified on labelled synthetic fixtures before any real camera is trusted.
- The gate camera turns the same pipeline into an automated entry/exit log, with plates masked by role for privacy.

SECTION 02 · SCOPE & STACK

2 Key Features and Technology Stack

Every row carries an explicit status, so implemented, disabled and unverified items are never confused. Evidence is detailed in Sections 7 and 8.

FEATURE	WHAT WAS BUILT	STATUS
Occupancy pipeline	YOLO11s detection, ByteTrack tracking, velocity gating and a per-slot state machine with measured debounce (0.6 s enter / 8.0 s leave).	IMPLEMENTED Observed on the real clip
Gate line-crossing & sessions	Entry/exit geometry, direction logic and track-to-plate binding.	IMPLEMENTED Verified on synthetic fixture replay
ANPR chain	Plate candidate detection, PP-OCRv4 OCR, format validation and per-track voting; 90 unit tests. Disabled in the shipped stream config: no recording with legible plates exists.	IMPLEMENTED · OFF BY DEFAULT NOT VALIDATED No real-plate accuracy
REST + WebSocket API	31 REST routes and 2 WebSocket routes under /api/v1; every route except login and refresh requires a token.	IMPLEMENTED
Security	Argon2id hashing, short-lived HS256 access tokens, refresh tokens with use tracking, epoch revocation, four roles, login throttling, per-role plate masking.	IMPLEMENTED
Live streaming	MJPEG camera feed with detection overlay, plus WebSocket events for slots and sessions.	IMPLEMENTED
Dashboard	SVG slot map, session table, live camera card and counters (React 19 + TypeScript).	IMPLEMENTED No dashboard screenshots included
Persistence	SQLite (WAL) at runtime; nine-table schema managed by Alembic; switchable to PostgreSQL by URL.	IMPLEMENTED
RTSP camera source	RTSP branch exists alongside file and webcam sources.	NOT VALIDATED Not tested on a physical camera
Docker / TLS deployment	Docker Compose services and nginx TLS configs exist; the stack has not been run end-to-end.	SCAFFOLDED · NOT RUN

Technology stack

LAYER	TECHNOLOGY	WHY IT WAS CHOSEN
Language	Python 3.10	Mature CV/ML ecosystem; type hints keep pipeline interfaces explicit.
Backend	FastAPI + Uvicorn	Async REST and WebSocket in one framework; Pydantic validation; auto docs.
Detection	Ultralytics YOLO11s (FP16, CUDA)	Good speed/accuracy trade-off; pretrained weights load without training.
Tracking	ByteTrack	Deterministic, low-cost track identity; enabled in Ultralytics by a config flag.
OCR / ANPR	rapidocr-onnxruntime (PP-OCRv4)	ONNX models ship inside the wheel: no second DL runtime, no weight download, no CUDA-provider conflict.
Vision I/O	OpenCV 4.13	Capture, geometry and rendering; version pinned to avoid dependency conflicts.
Database	SQLite + SQLAlchemy 2 + Alembic	Zero-admin runtime store; migrations make a later Postgres switch a URL change.
Frontend	React 19, TypeScript, Vite, Tailwind 4	Typed SPA with fast builds; TanStack Query for API state; Recharts for charts.
Testing	pytest, Vitest, oxlint	646 recorded backend tests; typed frontend logic tests; fast lint.
Deployment	Docker Compose + nginx (scaffold)	Reproducible multi-service deployment and TLS termination configs.

SECTION 03 · ARCHITECTURE

3 System Architecture

The system is built as **six layers**. Capture, perception and domain are pure Python with no HTTP or database imports — a guard test fails if that boundary breaks — so the intelligence chain runs headless and is fully unit-testable. Integration moves results onto an in-process event bus; persistence and delivery consume them.

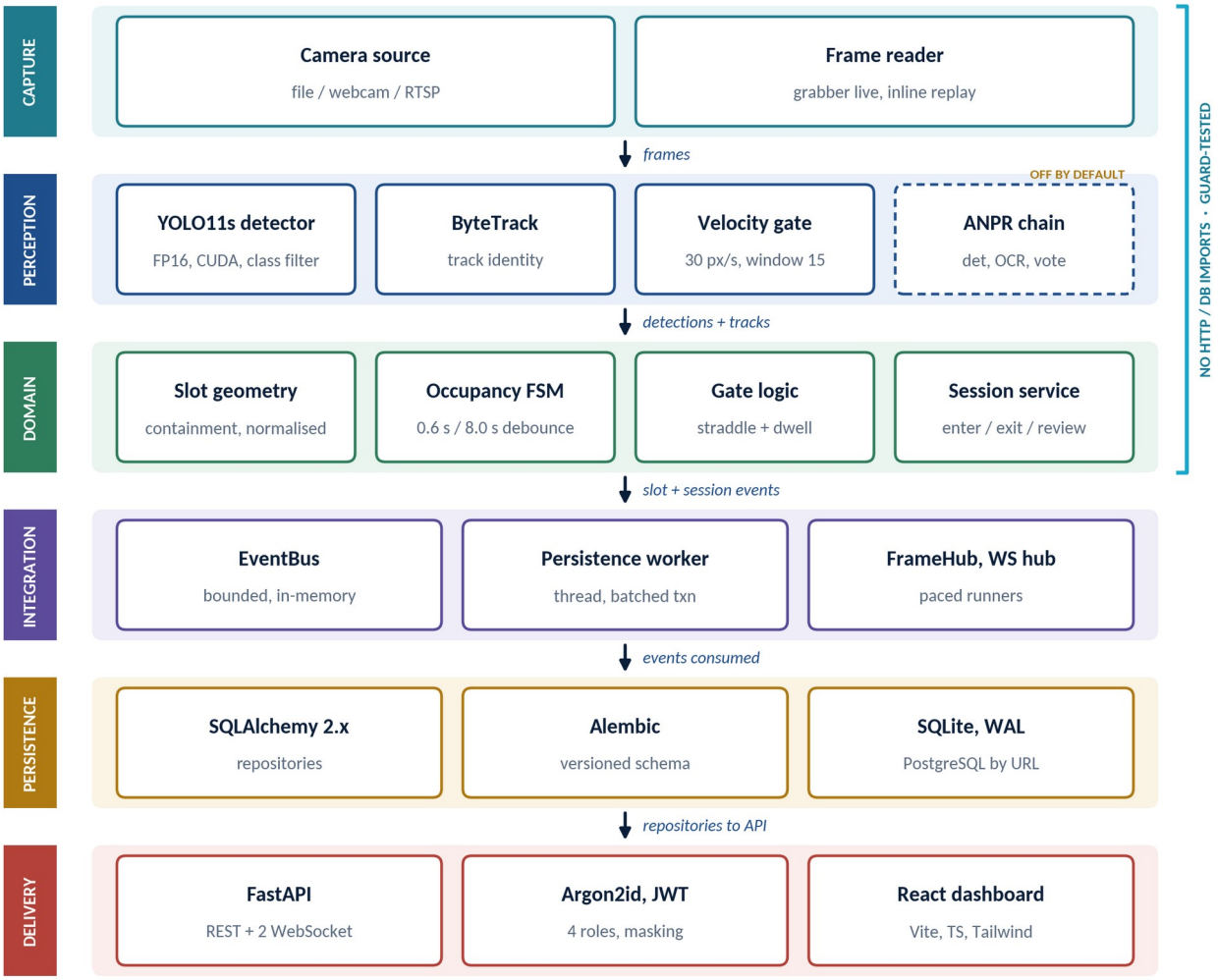


Figure 1 Layered architecture: capture, perception and domain logic are import-isolated from HTTP and persistence by guard tests.

How the components connect

CAPTURE	One reader per camera (file, webcam index or RTSP URL) paces frames into the pipeline; a replay mode feeds labelled clips frame-perfect for tests.
PERCEPTION	YOLO11s proposes vehicles, ByteTrack keeps identity, the velocity gate drops moving tracks from occupancy, and the ANPR chain turns gate crops into plate candidates.
DOMAIN	Calibrated slot polygons decide containment; a per-slot FSM absorbs jitter with 0.6 s / 8.0 s debounce; gate logic (straddle + dwell) and the session service own crossings.
INTEGRATION	Every decision becomes an event on a bounded, in-process `EventBus`. One worker thread drains it in batched transactions; the WebSocket hub serves per-viewer bounded queues; `FrameHub` keeps the latest frame per camera for MJPEG.
DELIVERY	FastAPI exposes the REST/WebSocket surface; the React dashboard subscribes to `/ws/live` and streams annotated MJPEG.

SECTION 04 · AI PIPELINE

4 AI and Computer-Vision Pipeline

Two independent paths run side by side and meet only at the event bus: the **lot path** answers "is this bay occupied?", the **gate path** answers "which vehicle crossed, when, and with what plate?". Keeping them separate means a gate failure can never corrupt occupancy, and each path is testable on its own.

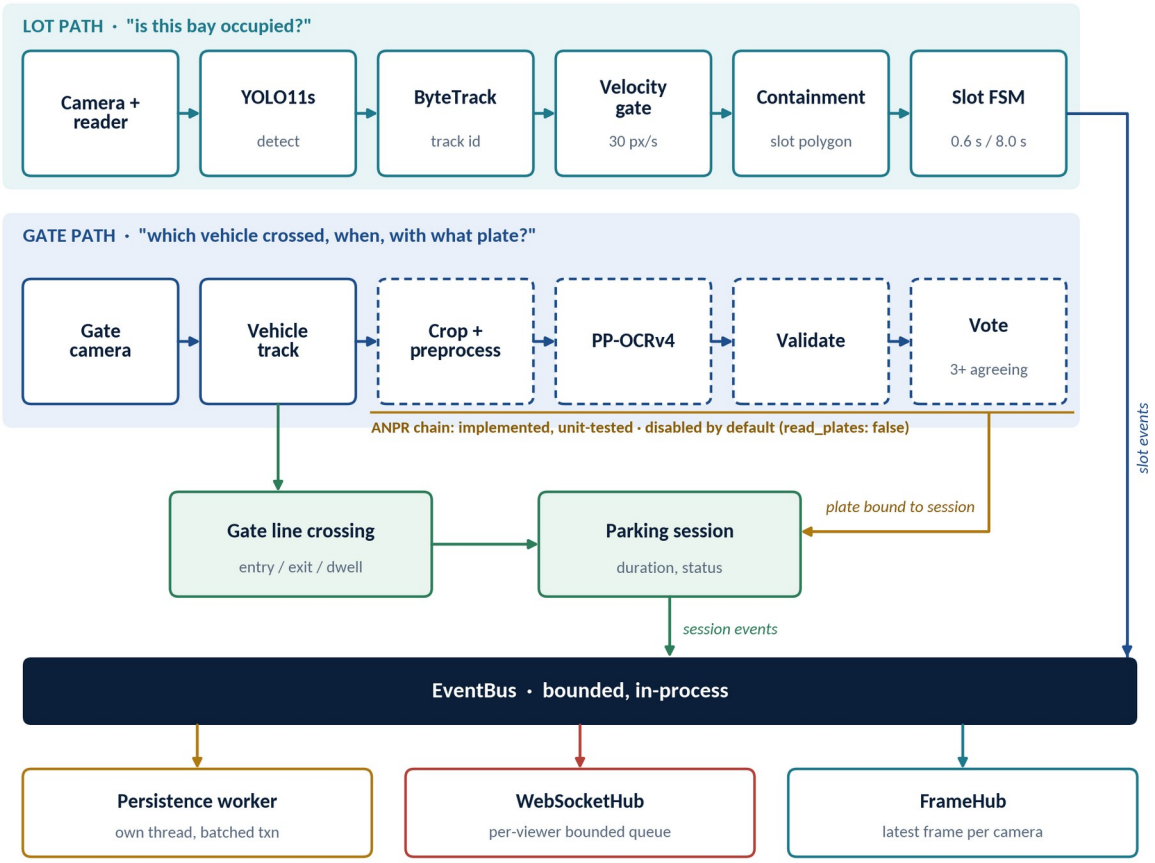


Figure 2 End-to-end data flow: the lot path (occupancy) and gate path (identity) run independently and converge only on the in-process event bus. Dashed boxes mark the ANPR stages, which are off by default.

Lot path — occupancy

- **Detection:** YOLO11s detects vehicles per frame (FP16 on CUDA, vehicle classes filtered).
- **Tracking:** ByteTrack carries identity across frames; the pipeline reasons about tracks, not per-frame detections.
- **Velocity gate:** a track whose median motion exceeds 30 px/s is driving, not parking — it cannot occupy a bay.
- **Containment:** each bay is a calibrated polygon; a track's normalised footprint must sit inside it.
- **Slot FSM:** a bay changes state only after 0.6 s of presence (enter) or 8.0 s of absence (leave); states are FREE / OCCUPIED / UNKNOWN.

Gate path — identity and sessions

- **Crossing:** the gate camera runs the same tracker; a crossing line with dwell logic decides entry versus exit direction.
- **Plate stage:** candidate regions inside the vehicle box (detector score, pixel size and shape screens) → crop and preprocessing → PP-OCRv4 → format validation (two Indian plate families plus configurable extra patterns).
- **Voting:** one read is never an answer; a plate is published only after 3+ agreeing reads inside a per-track window, then bound to the session.

> **ANPR STATUS — IMPLEMENTED, OFF BY DEFAULT**
>
> The chain is fully implemented and covered by 90 unit tests, but it is switched off in the shipped configuration (`read_plates: false`) because no recording with legible plates exists on the development machine. All recognition numbers quoted in this report come from **rendered plate fixtures** and are labelled as such — they characterise the software chain, not a real camera.

SECTION 05 · BACKEND & API

5 Backend, Database and API

Backend architecture

- **FastAPI app** assembled from routers (auth, cameras, slots, sessions, plates, stats, audit, health); dependencies inject configuration, the DB session and role checks.
- **Persistence worker** (own thread): drains the event bus in batches of up to 200; a failed event is retried; at most 200 failure samples are kept, so one bad row never blocks the stream. A browser request or a commit never blocks inference.
- **SQLite** is the runtime store (WAL mode, single-writer guard that refuses a second process); the same models run on PostgreSQL by setting the database URL.

Database — important entities

Nine tables, versioned by Alembic migrations 0001–0003 (including the token-revocation epoch column). Sessions join crossing events to plates; occupancy history preserves slot state changes over time; the audit log records every administrative action.

`users`	`cameras`	`parking_slots`
`vehicles`	`license_plates`	`parking_sessions`
`slot_occupancy_history`	`detection_logs`	`audit_logs`

API surface

GROUP	ROUTES	PURPOSE
Auth & users	9	Login, refresh, logout, WebSocket ticket, user management
Cameras	8	List/status, restart, snapshot, annotated MJPEG stream
Slots	4	Slot map, summary, polygon update
Sessions	4	List, detail, close, plate correction
Plates	1	Masked search (minimum 3 normalised characters)
Stats	3	Occupancy series, durations; revenue answers 501 (not implemented)
Audit & health	2	Audit log page, service health
WebSocket	2	`/ws/live` events, `/ws/cameras/{id}/overlay` frame data

All REST routes live under `/api/v1`; the WebSocket routes are mounted at the root. Every route except `POST /auth/login` and `POST /auth/refresh` carries a token dependency.

Authentication, authorization and security

- **Passwords:** Argon2id hashing with rehash-on-login.
- **Tokens:** short-lived HS256 access tokens (900 s) and refresh tokens (43 200 s) with single-use tracking.
- **Revocation:** a per-user epoch timestamp invalidates every token issued before it (migration 0003).
- **Roles:** ADMIN, OPERATOR, VIEWER and GUEST; write operations are restricted to ADMIN/OPERATOR.
- **Privacy:** plates are masked for non-privileged roles (`TS09DP4567` → `TS09**4567`); access logs mask tokens and tickets.
- **Hardening:** login throttle (10 failures / 300 s per address), audit entries for every action, single-writer database guard with 12 dedicated tests.

SECTION 06 · FRONTEND

6 Frontend and User Workflow

Dashboard

- SVG slot map of calibrated bays, colour-coded by state and updated by WebSocket.
- Live camera card with the annotated MJPEG picture and a socket status chip reading "Live".
- Session table with masked plate, entry/exit times and status.
- Counter row: total, occupied, free.
- TanStack Query manages API state; Recharts renders occupancy and session charts.

User workflow

1	A camera frame is paced by the frame grabber into the pipeline.
2	Detection, tracking and the slot FSM produce occupancy and session events on the event bus.
3	The worker persists them; hubs fan them out — latest frame per camera, per-viewer socket queues.
4	The dashboard signs in (JWT), loads state over REST, then subscribes to `/ws/live`.
5	The slot map, counters and session table update live; the camera card streams annotated MJPEG.



Figure 3 Annotated frame from the live lot camera (`lot_cam1`, real street clip) as served by the MJPEG stream: detection boxes, ByteTrack ids, calibrated bay polygons and the occupancy HUD. The registration plate on the car is covered by an opaque bar; the pipeline's plate output is disabled for this stream.

EVIDENCE NOTE

This frame is the genuine visual evidence captured during development. **No dashboard screenshots are included:** producing them would require starting the server against the live database, which this documentation intentionally does not do.

SECTION 07 · VERIFICATION

7 Implementation and Testing

What was implemented

PHASES 1–4	PHASE 5	PHASE 6	PHASES 7–8
Camera capture and frame reader; detection, tracking and rendering; slot geometry and the occupancy FSM; persistence and the first REST API.	The full ANPR chain: detection, OCR, validation and voting.	Gate lines, direction logic and parking sessions.	WebSockets and the live dashboard; authentication, audit and hardening (revocation, throttling, one-writer guard).

Test evidence

SUITE	RECORDED RESULT	EVIDENCE
Backend pytest	646 passed in 85.38 s (2026-10-14)	RECORDED RUN recorded pytest output
Unit tests (static)	535 test functions across 26 modules	STATIC COUNT static count of the current tree
ANPR	90 tests passing	UNIT TESTS validation, voting, association, pipeline, config
Frontend	53 logic tests recorded; 7 `*.test.ts` files in tree	RECORDED RUN Vitest run + recorded build log
Production build	632.42 kB / 189.33 kB gzip	RECORDED RUN frontend build log

Real footage versus synthetic fixtures

REAL FOOTAGE

- One real recording in the repository: a street clip served by `lot_cam1`.
- Validates the occupancy chain **by observation**.
- All **748 vehicle tracks** stayed below the ANPR plate-width floor, so the plate stage correctly produced no candidates.
- No accuracy score is claimed for real plates; no mAP/benchmark exists for the detector.

SYNTHETIC FIXTURE

- Gate acceptance replay, ANPR end-to-end replay and a rendered plate fixture (280 plates).
- Labelled clips and rendered plates prove the **software chain**, reproducibly, on any machine.
- They characterise the software chain, **not** a real camera or gate deployment.
- Every fixture output is labelled SYNTHETIC.

Synthetic-fixture results

TEST	RESULT
Gate acceptance replay	5 of 5 fixture vehicles behaved exactly as designed; 13 of 13 events handled, 0 dropped; 12 transactions → 4 sessions, 6 plates, 6 vehicles.
ANPR end-to-end replay	Rendered clip with a ground-truth plate: all 24 accepted reads correct; the plate stable after three agreeing reads at 1.20 s; exactly one event published.
Rendered plate fixture	280 plates, 30–200 px wide: exact reads 2 % at 30 px rising to 55 % at 150 px. By degradation — noise 41 %, clean 32 %, compressed 32 %, blur 20 %, tilt 16 %.
Measured cost	Plate detector ≈ 30 ms/vehicle, OCR ≈ 14 ms/crop, 6.1 fps CPU-only on one gate stream; geometry-only fixture replay at 2 638 fps overall.

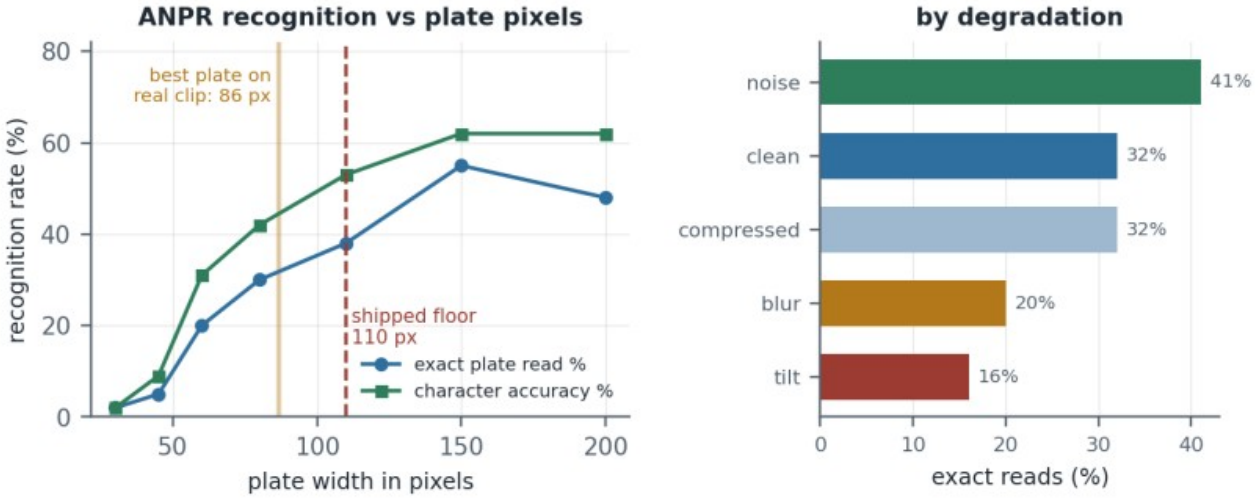


Figure 4 ANPR fixture results: exact-match reads by rendered plate width and by degradation type, measured with `bench_anpr.py`. Synthetic fixture — it validates the software chain, not a real camera.

VERIFICATION BOUNDARY

No accuracy score is claimed for real plates, and no mAP/benchmark exists for the detector — decisions were verified by observation on the real clip and by labelled fixtures for the software chain.

SECTION 08 · RISKS & ROADMAP

8 Challenges, Limitations and Future Improvements

Major technical challenges

CHALLENGE	WHAT WAS FOUND
Plate resolution	The binding constraint is the plate size, not the model: below ≈ 110 px wide, exact reads collapse (2–5 % at 30–45 px). The pipeline rejects silently instead of guessing.
Hybrid-CPU scheduling	Identical OCR inference measured ≈ 11–14 ms on P-cores versus ≈ 150 ms after Windows migrated the threads to E-cores — a scheduling effect, not model cost. Measured and documented rather than "fixed".
One-writer SQLite	Two processes writing the same file corrupt it, so the server refuses to start when another writer is detected (guard plus 12 tests).
Real camera access	The development machine's live sources are video files; the RTSP branch exists but has not been tested against a physical camera.
Layering discipline	Keeping capture, perception and domain import-free of HTTP and persistence — the constraint that keeps 600+ tests fast and honest.

Current limitations

LIMITATION	DETAIL	STATUS
ANPR not enabled	Implemented but disabled in the shipped config; the plate tables hold zero rows and no field accuracy is measured.	OFF BY DEFAULT · NOT VALIDATED
No 24-hour soak	Longest recorded green run ≈ 11 minutes, which exercised PostgreSQL; the SQLite profile has no soak evidence. A corruption incident reported during development is only partly evidenced; the one-writer guard now prevents recurrence.	NOT VALIDATED
Deployment unverified	Docker/TLS is scaffolded, not verified running: only the database container was observed up (2026-10-14); `deploy/ssl/` and `deploy/backups/` are empty.	SCAFFOLDED · NOT RUN
Revenue / billing	The revenue endpoint returns 501; billing and payment are out of scope.	NOT IMPLEMENTED
Login throttle	In-memory per process, with no account lockout.	PARTIAL
Plate formats	Two-line and heavily oblique plates are excluded.	BY DESIGN

Practical next steps

01
ANPR on real footage
Record a gate camera with legible plates → enable ANPR on that stream → measure real per-frame and voted accuracy.
PLANNED

02
Real RTSP camera
Wire a real RTSP camera and re-calibrate the slot polygons for the physical site.
PLANNED

03
Deployment and soak
Run the Docker Compose TLS stack end-to-end and execute a real 24-hour soak on the SQLite profile.
PLANNED

04
Product features
Billing module (sessions and schema already exist); persistent throttle with account lockout; a mobile-friendly dashboard view.
PLANNED

SECTION 09 · OUTCOME

9 Conclusion and Project Links

What the project currently achieves

An end-to-end chain from camera pixel to live dashboard: detection, tracking, occupancy with measured debounce, gate sessions, an ANPR stage (off by default), an audited role-based API and a live UI — backed by **646 passing backend tests** and honest, fixture-labelled verification. The parts that are not yet validated (real-plate accuracy, a live Docker deployment, a 24-hour soak) are named as such rather than assumed.

Evidence at a glance

VERIFIED

- 646 backend tests passed in a recorded run (2026-10-14); 90 ANPR unit tests.
- Occupancy chain observed on the one real street clip (748 vehicle tracks).
- Synthetic-fixture replays: gate acceptance 5 of 5 vehicles; ANPR end-to-end 24 of 24 accepted reads correct.
- Single-writer database guard covered by 12 dedicated tests.

NOT YET VALIDATED

- ANPR accuracy on real plates (stage disabled by default).
- Live Docker Compose / TLS deployment.
- A 24-hour soak, in particular on the SQLite profile.
- RTSP input against a physical camera.
- Detector mAP / benchmark (none exists).

Key engineering decisions and learnings

Fixture-first verification

Rendered plates and labelled clips prove the software chain before any real camera is trusted; every fixture output is labelled SYNTHETIC.

Measured choices

RapidOCR over EasyOCR/PaddleOCR (dependency conflicts were measured, not preferred); the OCR scaling mode chosen by measured fixture recall; debounce values measured, not guessed.

Strict layering pays

Guard-tested import boundaries made the intelligence chain testable without mocking the world.

Privacy by construction

Per-role plate masking, masked access logs, and redacted reproductions of real footage.

Project links

GITHUB REPOSITORY	https://github.com/saurabh876724/Smart-parking.git
LIVE DEMO	No public deployment exists; the repository README contains the full setup, run and fixture commands.

FOR REVIEWERS — WHERE TO LOOK

If you are reviewing this as a recruiter or fellow developer, the interesting parts are `src/smart_parking/parking`, `anpr` and `pipeline` (the intelligence), the guard tests that keep them pure, and the fixture suite that makes their behaviour reproducible on any machine.